OOPS with Java
Mid-Semester Study Material

Prepared from the March 2026 mid-semester paper

Contents

1 How to Use This Guide **1**

2 Roadmap from Paper to Study Topics **1**

3 Prerequisite Concepts **2**
- 3.1 What is a program? . . . 2
- 3.2 What is a class? . . . 2
- 3.3 What is an object? . . . 2
- 3.4 Data members and methods . . . 3
- 3.5 Access modifiers . . . 3
- 3.6 Constructors . . . 3
- 3.7 Arrays . . . 3
- 3.8 Loops and decision making . . . 4
- 3.9 Static and non-static members . . . 4
- 3.10 What is `null`? . . . 4

4 Core OOP Principles **5**
- 4.1 Encapsulation . . . 5
- 4.2 Abstraction . . . 5
- 4.3 Inheritance . . . 5
- 4.4 Polymorphism . . . 6

5 Object References and Equality **7**
- 5.1 Primitive values vs object references . . . 7
- 5.2 `==` operator . . . 7
- 5.3 `equals()` method . . . 7
- 5.4 Example . . . 7
- 5.5 Common mistake . . . 7

6 Inheritance Problems and the Diamond Problem **8**
- 6.1 What is the diamond problem? . . . 8
- 6.2 Why Java avoids it for classes . . . 8
- 6.3 What about interfaces? . . . 8

7 Abstract Class and Interface **9**
7.1 Why do we need abstraction? . . . 9
7.2 Abstract class . . . 9
7.3 Interface . . . 9
7.4 Side-by-side understanding . . . 9
7.5 Example with both . . . 10

8 Keyword Guide: this, super, final **10**
8.1 `this` . . . 10
8.2 `super` . . . 11
8.3 `final` . . . 11

9 Method Overloading and Method Overriding **12**
9.1 Method overloading . . . 12
9.2 Method overriding . . . 12
9.3 Why both matter in the paper . . . 12

10 Exceptions and Exception Handling **12**
10.1 What is an exception? . . . 12
10.2 Why use exceptions? . . . 13
10.3 Basic keywords . . . 13
10.4 Example . . . 13
10.5 User-defined exception . . . 13
10.6 Flow of execution for a thrown exception . . . 13

11 Nested Classes **14**
11.1 What is a nested class? . . . 14
11.2 Static nested class . . . 14
11.3 Inner class . . . 14
11.4 How to create objects . . . 15
11.5 Why this topic is important . . . 15

12 Understanding the Error-Finding Code Question **15**
12.1 Given ideas in the paper . . . 15
12.2 Main rules you must remember . . . 15
12.3 Correct mental model . . . 16

13 Study Material for the Student Grade Program **16**
13.1 Problem understanding . . . 16
13.2 How to think before coding . . . 16
13.3 Program . . . 17
13.4 Why this is a good answer . . . 18

14 Study Material for the this, super, final Question **18**
14.1 `this`: very common beginner confusion . . . 18
14.2 `super`: child talks to parent . . . 18
14.3 `final`: three interpretations . . . 19

15 Study Material for Custom Exceptions **19**

15.1 When should we create a custom exception? . . . 19
15.2 Example: Invalid age . . . 19
15.3 What happens when `throw` runs? . . . 19
15.4 Complete demo . . . 20

16 System Design Questions: How to Approach Them **20**
16.1 What the examiner expects . . . 20
16.2 A reliable solving strategy . . . 20

17 Bank Management System Study Material **21**
17.1 Requirement breakdown . . . 21
17.2 Design thinking . . . 21
17.3 Program . . . 21
17.4 What concepts are shown here? . . . 22

18 Library Management System Study Material **22**
18.1 Requirement breakdown . . . 22
18.2 Design thinking . . . 23
18.3 Program . . . 23
18.4 What concepts are shown here? . . . 24

19 How to Write Better Answers in the Exam **24**
19.1 For theory questions . . . 24
19.2 For difference questions . . . 25
19.3 For code tracing or error questions . . . 25
19.4 For program design questions . . . 25

20 Common Mistakes to Avoid **25**

21 Quick Revision Notes **26**

22 Self-Check Questions **26**

23 Final Advice **27**

Purpose of This Material

This guide is designed for a student who wants to understand every concept required by the mid-semester paper, even if the student is still weak in Java fundamentals. The material is written in a prerequisite-first order. That means we first build the basic ideas such as classes, objects, methods, arrays, and access modifiers, and then move to exam topics such as abstraction, interfaces, nested classes, exceptions, and mini system design questions.

1 How to Use This Guide

1. Read the sections in order at least once.
2. Do not jump directly to the long design questions unless you already understand classes, constructors, arrays, interfaces, and exceptions.

3. For every code example, ask:
   - What is the class?
   - What is the object?
   - Which data belongs to the object?
   - Which method changes or uses that data?
4. After each major topic, try to explain it in your own words using one small example.

2 Roadmap from Paper to Study Topics

Question	**Main concept**	**What you must know first**
Q1	OOP principles	Class, object, data, method
Q2	`==` and `equals()`	Primitive types, objects, references, `String`
Q3	Diamond problem	Inheritance, method ambiguity, interfaces
Q4	Abstract class vs interface	Inheritance, overriding, abstraction
Q5	Errors in nested class code	Static vs non-static, object creation, inner class rules, `null`
Q6	Student grading program	Arrays, loops, methods, class design, if-else
Q7	`this`, `super`, `final`	Inheritance, constructors, current object
Q8	Custom exception / nested classes	try-catch, `throw`, `throws`, static and inner classes
Q9	Bank / library system design	Encapsulation, constructors, overloading, overriding, interfaces, exceptions

3 Prerequisite Concepts

3.1 What is a program?

A program is a set of instructions given to the computer. In Java, we write those instructions inside classes and methods. Java programs are usually organized in a more structured way than simple procedural programs. Instead of keeping data and behavior separate, Java encourages us to keep them together inside classes.

3.2 What is a class?

A class is a blueprint. It describes:

- what data an object will store
- what actions that object can perform

Example idea: A `Student` class may store roll number, name, and marks, and may contain methods to calculate total and percentage.

```
class Student {
    int rollNo;
```

```
    String name;

    void display() {
        System.out.println(rollNo + " " + name);
    }
}
```

3.3 What is an object?

An object is a real instance created from a class.

```
Student s1 = new Student();
```

Here:

- `Student` is the class
- `s1` is a reference variable
- `new Student()` creates the object

3.4 Data members and methods

- **Data members** store information about the object.
- **Methods** define what the object can do.

Example:

```
class Fan {
    int speed;
    boolean isOn;

    void start() {
        isOn = true;
    }
}
```

3.5 Access modifiers

Access modifiers control where a variable or method can be used.

- `public`: accessible everywhere
- `private`: accessible only inside the same class
- `protected`: accessible in the same package and subclasses
- default access: accessible in the same package

Why important? Because many OOP exam questions are about encapsulation, and encapsulation depends on access control.

3.6 Constructors

A constructor initializes an object. It has the same name as the class and no return type.

```
class Book {
    int id;
    String title;

    Book(int id, String title) {
        this.id = id;
        this.title = title;
    }
}
```

3.7 Arrays

An array stores multiple values of the same type.

```
int[] marks = new int[5];
marks[0] = 80;
marks[1] = 90;
```

This is a key prerequisite for the student grading question because marks for 5 subjects are naturally stored in an array.

3.8 Loops and decision making

- `for` loop helps process arrays.
- `if-else` helps make decisions such as assigning grades.

```
int total = 0;
for (int i = 0; i < marks.length; i++) {
    total += marks[i];
}

if (total >= 400) {
    System.out.println("Good score");
} else {
    System.out.println("Needs improvement");
}
```

3.9 Static and non-static members

This is one of the most important prerequisites for Q5 and Q8.

- A non-static member belongs to an object.
- A static member belongs to the class itself.

Example:

```java
class Demo {
    int x = 10; // instance variable
    static int y = 20; // class variable
}
```

Key rule:

- Static methods can directly use only static data.
- To access non-static data inside a static context, you need an object.

3.10 What is `null`?

`null` means a reference variable does not point to any object.

```java
Student s = null;
```

If we try to use `s.someMethod()` when `s` is `null`, Java throws `NullPointerException`.

4 Core OOP Principles

4.1 Encapsulation

Encapsulation means wrapping data and methods together inside a class and protecting the data from direct misuse.

Bad design:

```java
class Account {
    public double balance;
}
```

Anyone can change the balance to any wrong value.

Better design:

```java
class Account {
    private double balance;

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    public double getBalance() {
        return balance;
    }
}
```

Why it matters:

- protects data

- prevents invalid object state
- improves maintainability

4.2 Abstraction

Abstraction means showing only the essential part and hiding unnecessary internal details.

Real-life analogy: When you drive a car, you use steering, brake, and accelerator. You do not need to understand the engine internals every time you drive.

In Java:

- abstract classes
- interfaces

4.3 Inheritance

Inheritance allows one class to reuse data and behavior of another class.

```
class Animal {
    void eat() {
        System.out.println("Animal eats");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}
```

Now `Dog` can use both `eat()` and `bark()`.

4.4 Polymorphism

Polymorphism means one interface, many forms. In Java, it mainly appears as:

- **method overloading** - same method name, different parameter lists
- **method overriding** - child class gives a new implementation of a parent method

Overloading example:

```
class PrintDemo {
    void show(int x) {
        System.out.println(x);
    }

    void show(String s) {
        System.out.println(s);
    }
}
```

Overriding example:

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Bark");
    }
}
```

5 Object References and Equality

5.1 Primitive values vs object references

Before understanding `==` and `equals()`, we must know the difference between primitives and objects.

Primitive example:

```
int a = 5;
int b = 5;
```

Here, `a == b` compares values directly.

Object example:

```
String s1 = new String("Java");
String s2 = new String("Java");
```

Here, `s1` and `s2` are references.

5.2 == operator

- For primitives: compares actual value
- For objects: compares memory reference

5.3 `equals()` method

`equals()` compares logical content if the class defines meaningful comparison. `String` overrides `equals()` to compare actual characters.

5.4 Example

```
public class CompareDemo {
    public static void main(String[] args) {
        String a = new String("OOP");
        String b = new String("OOP");
```

```
        System.out.println(a == b); // false
        System.out.println(a.equals(b)); // true
    }
}
```

5.5 Common mistake

Students often use == for string comparison:

```
if (name == "Java") // not recommended for content comparison
```

The better approach is:

```
if (name.equals("Java"))
```

6 Inheritance Problems and the Diamond Problem

6.1 What is the diamond problem?

Suppose a language allows this:

- class A has method `show()`
- class B and class C both inherit from A
- class D inherits from both B and C

Now class D gets two possible paths to method `show()`. Which one should it use? This ambiguity forms a diamond-like inheritance diagram, so it is called the diamond problem.

6.2 Why Java avoids it for classes

Java does not allow multiple inheritance of classes:

```
class A { }
class B extends A { }
class C extends A { }
// class D extends B, C { } // not allowed in Java
```

This simple rule removes ambiguity for class inheritance.

6.3 What about interfaces?

Java allows a class to implement multiple interfaces. If two interfaces have the same `default` method, the implementing class must resolve it explicitly.

```
interface A {
    default void show() {
        System.out.println("A");
    }
}
```

```java
interface B {
    default void show() {
        System.out.println("B");
    }
}

class C implements A, B {
    @Override
    public void show() {
        System.out.println("Resolved in C");
    }
}
```

Conclusion:

- No multiple inheritance of classes
- Multiple interfaces allowed
- Conflicting default methods must be resolved by overriding

7 Abstract Class and Interface

7.1 Why do we need abstraction?

Sometimes we want to define a general idea without giving full implementation at once.

Example: Different payment methods all support `pay()`, but their implementations differ.

7.2 Abstract class

An abstract class is a partially complete class.

```java
abstract class Shape {
    abstract double area();

    void message() {
        System.out.println("Area calculation");
    }
}
```

- can contain abstract methods
- can contain normal methods
- can have instance variables
- can have constructors

7.3 Interface

An interface defines a contract.

```java
interface Printable {
    void print();
}
```

- used when different classes must follow the same rule
- supports multiple implementation
- ideal for capability-based design

7.4 Side-by-side understanding

Point	Abstract class	Interface
Nature	Partial implementation	Contract only
Fields	Can have instance fields	Only constants by default
Constructors	Yes	No
Inheritance	Single inheritance only	Multiple interfaces can be implemented
Best use	Shared base code	Common behavior across unrelated classes

7.5 Example with both

```java
abstract class Vehicle {
    String brand;

    Vehicle(String brand) {
        this.brand = brand;
    }

    abstract void start();
}

interface Electric {
    void charge();
}

class Car extends Vehicle implements Electric {
    Car(String brand) {
        super(brand);
    }

    @Override
    void start() {
        System.out.println("Car starts");
    }

    @Override
    public void charge() {
        System.out.println("Car is charging");
    }
```

```
}
```

8 Keyword Guide: this, super, final

8.1 this

`this` refers to the current object.

Main uses:

- to distinguish instance variable from local variable
- to call another constructor of the same class
- to pass the current object

```
class Student {
    int rollNo;

    Student(int rollNo) {
        this.rollNo = rollNo;
    }
}
```

8.2 super

`super` refers to the immediate parent class object.

Main uses:

- call parent constructor
- access parent method
- access parent variable if hidden by child variable

```
class A {
    void show() {
        System.out.println("A");
    }
}

class B extends A {
    void show() {
        super.show();
        System.out.println("B");
    }
}
```

8.3 `final`

`final` means "cannot be changed further".

Three common uses:

1. **final variable:** value cannot be reassigned
2. **final method:** cannot be overridden
3. **final class:** cannot be inherited

```
final class Utility { }

class Demo {
    final int x = 10;

    final void show() {
        System.out.println("Fixed behavior");
    }
}
```

9 Method Overloading and Method Overriding

9.1 Method overloading

Same method name, different parameter lists, usually inside the same class.

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }
}
```

9.2 Method overriding

A child class changes the implementation of a parent class method.

```
class Animal {
    void sound() {
        System.out.println("Generic sound");
    }
}

class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Meow");
    }
}
```

9.3 Why both matter in the paper

- Bank Management System asks for constructor and method overloading, and also overriding for interest calculation.
- Library Management System asks for overriding where required and overloaded `issueBook()` methods.

10 Exceptions and Exception Handling

10.1 What is an exception?

An exception is an abnormal event that interrupts normal program flow. Instead of crashing blindly, Java allows us to handle such situations properly.

Examples:

- dividing by zero
- accessing a null reference
- invalid age for a rule
- book not available
- insufficient balance

10.2 Why use exceptions?

Without exceptions, programs become difficult to manage because error cases mix with normal logic everywhere.

10.3 Basic keywords

- `try`: risky code
- `catch`: error handling code
- `finally`: code that runs whether exception occurs or not
- `throw`: used to throw an exception explicitly
- `throws`: used in method declaration

10.4 Example

```java
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
}
```

10.5 User-defined exception

We create a custom exception when built-in exceptions do not clearly express the business rule.

```
class InvalidAgeException extends Exception {
    InvalidAgeException(String message) {
        super(message);
    }
}
```

10.6 Flow of execution for a thrown exception

1. Program enters `try`
2. Some method detects invalid condition
3. `throw new ExceptionType(...)` runs
4. Normal flow stops at that point
5. Control moves to matching `catch`
6. Program continues after catch

11 Nested Classes

11.1 What is a nested class?

A nested class is a class declared inside another class.

Types:

1. static nested class
2. member inner class
3. local inner class
4. anonymous inner class

11.2 Static nested class

- declared with `static`
- belongs to the outer class, not to an object
- can directly access only static members of the outer class

```
class Outer {
    static int y = 10;

    static class Nested {
        void show() {
            System.out.println(y);
```

```
        }
    }
}
```

11.3 Inner class

- non-static class inside another class
- belongs to an object of the outer class
- can access both static and non-static members of the outer class

```
class Outer {
    int x = 20;

    class Inner {
        void show() {
            System.out.println(x);
        }
    }
}
```

11.4 How to create objects

Static nested class:

```
Outer.Nested obj = new Outer.Nested();
```

Inner class:

```
Outer o = new Outer();
Outer.Inner in = o.new Inner();
```

11.5 Why this topic is important

This topic appears in two ways in the paper:

- direct theory/program question on nested classes
- error-finding question where nested class rules are violated

12 Understanding the Error-Finding Code Question

12.1 Given ideas in the paper

The question tests:

- static nested class rules
- inner class rules
- object creation rules

- compile-time error vs runtime error

12.2 Main rules you must remember

1. A static nested class cannot directly use a non-static outer instance variable.
2. A non-static inner class cannot generally declare a normal static variable.
3. A non-static inner class object requires an outer object.
4. Calling a method that does not exist causes compile-time error.
5. Creating an inner object using a null outer reference causes runtime failure.

12.3 Correct mental model

- **Outer object required?** If yes, you are dealing with a non-static inner class.
- **Can it access outer instance data directly?** Static nested class: no. Inner class: yes.
- **Can static context see object data?** Not without an object.

13 Study Material for the Student Grade Program

13.1 Problem understanding

We need a program that:

- stores roll number and name
- accepts marks for 5 subjects
- calculates total
- calculates percentage
- assigns grade
- displays the result

13.2 How to think before coding

Step 1: Identify data

- roll number -> `int`
- name -> `String`
- marks for 5 subjects -> `int[]`
- total -> `int`
- percentage -> `double`

- grade -> `char` or `String`

Step 2: Identify actions

- accept details
- calculate total and percentage
- assign grade
- display result

13.3 Program

```
import java.util.Scanner;

class Student {
    private int rollNumber;
    private String name;
    private int[] marks = new int[5];
    private int total;
    private double percentage;
    private char grade;

    void acceptDetails() {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter roll number: ");
        rollNumber = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter name: ");
        name = sc.nextLine();

        for (int i = 0; i < marks.length; i++) {
            System.out.print("Enter marks of subject " + (i + 1) + ": ");
            marks[i] = sc.nextInt();
        }
    }

    void calculateTotalAndPercentage() {
        total = 0;
        for (int mark : marks) {
            total += mark;
        }
        percentage = total / 5.0;
    }

    void assignGrade() {
        if (percentage >= 90) grade = 'A';
        else if (percentage >= 80) grade = 'B';
        else if (percentage >= 70) grade = 'C';
        else if (percentage >= 60) grade = 'D';
        else grade = 'F';
```

```
    }

    void displayResult() {
        System.out.println("Roll No : " + rollNumber);
        System.out.println("Name : " + name);
        System.out.println("Total : " + total);
        System.out.println("Percentage: " + percentage);
        System.out.println("Grade : " + grade);
    }
}

public class StudentResultDemo {
    public static void main(String[] args) {
        Student s = new Student();
        s.acceptDetails();
        s.calculateTotalAndPercentage();
        s.assignGrade();
        s.displayResult();
    }
}
```

13.4 Why this is a good answer

- Uses a class correctly
- Uses array for marks
- Separates logic into methods
- Follows the exact requirement of the question

14 Study Material for the this, super, final Question

14.1 `this`: very common beginner confusion

Consider:

```
class Book {
    String title;

    Book(String title) {
        this.title = title;
    }
}
```

Why is `this.title` needed? Because the parameter name and instance variable name are same. Without `this`, Java will treat `title` as the local parameter.

14.2 `super`: child talks to parent

```
class Person {
    String name;
```

```
    Person(String name) {
        this.name = name;
    }
}

class Student extends Person {
    int rollNo;

    Student(String name, int rollNo) {
        super(name);
        this.rollNo = rollNo;
    }
}
```

`super(name)` sends the name to the parent constructor.

14.3 `final`: three interpretations

- final variable -> fixed value
- final method -> no overriding
- final class -> no inheritance

Memory trick: `final` means "stop further change".

15 Study Material for Custom Exceptions

15.1 When should we create a custom exception?

When the error is related to our business rule, not just Java syntax.

Examples:

- invalid age
- insufficient balance
- book not available

15.2 Example: Invalid age

```
class InvalidAgeException extends Exception {
    InvalidAgeException(String message) {
        super(message);
    }
}

class AgeCheck {
    static void check(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Age must be at least 18");
```

```
        }
        System.out.println("Eligible");
    }
}
```

15.3 What happens when `throw` runs?

As soon as `throw` is executed:

- the current method stops normal execution
- Java looks for a matching catch block
- if found, control goes there

15.4 Complete demo

```
public class ExceptionDemo {
    public static void main(String[] args) {
        try {
            AgeCheck.check(16);
            System.out.println("This line will not run if exception occurs");
        } catch (InvalidAgeException e) {
            System.out.println("Handled: " + e.getMessage());
        }

        System.out.println("Program continues");
    }
}
```

16 System Design Questions: How to Approach Them

16.1 What the examiner expects

For design questions, the examiner usually checks whether you can:

- identify classes correctly
- choose suitable data members
- write constructors
- use inheritance or interfaces where required
- use overloading and overriding properly
- handle exceptional cases
- demonstrate the program in `main()`

16.2 A reliable solving strategy

1. Read the requirement and underline nouns and actions.
2. Convert nouns into classes and data members.
3. Convert actions into methods.
4. See whether the question explicitly demands interface, inheritance, overloading, overriding, or exception handling.
5. Build the smallest correct design first.

17 Bank Management System Study Material

17.1 Requirement breakdown

The question asks for:

- class `Account`
- constructor overloading
- method overloading
- class `SavingsAccount` extending `Account`
- interface `Transaction`
- exception for insufficient balance

17.2 Design thinking

- `Account` should hold account number, holder name, balance
- `Transaction` should contain `deposit()` and `withdraw()`
- `SavingsAccount` should override the interest method

17.3 Program

```
class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String message) {
        super(message);
    }
}

interface Transaction {
    void deposit(double amount);
    void withdraw(double amount) throws InsufficientBalanceException;
}

class Account implements Transaction {
    protected int accountNumber;
```

```java
    protected String holderName;
    protected double balance;

    Account() {
        this(0, "Unknown", 0.0);
    }

    Account(int accountNumber, String holderName) {
        this(accountNumber, holderName, 0.0);
    }

    Account(int accountNumber, String holderName, double balance) {
        this.accountNumber = accountNumber;
        this.holderName = holderName;
        this.balance = balance;
    }

    @Override
    public void deposit(double amount) {
        balance += amount;
    }

    @Override
    public void withdraw(double amount) throws InsufficientBalanceException {
        if (amount > balance) {
            throw new InsufficientBalanceException("Insufficient balance");
        }
        balance -= amount;
    }

    double calculateInterest() {
        return balance * 0.04;
    }
}

class SavingsAccount extends Account {
    SavingsAccount(int accountNumber, String holderName, double balance) {
        super(accountNumber, holderName, balance);
    }

    @Override
    double calculateInterest() {
        return balance * 0.06;
    }
}
```

17.4 What concepts are shown here?

- **Encapsulation**: data grouped in classes
- **Constructor overloading**: multiple constructors in `Account`
- **Inheritance**: `SavingsAccount extends Account`

- **Overriding**: changed interest calculation
- **Interface**: transaction contract
- **Exception handling**: insufficient balance case

18 Library Management System Study Material

18.1 Requirement breakdown

The question asks for:

- class `Book`
- class `Member`
- encapsulation and constructors
- interface `Issueable`
- method `issueBook()`
- exception if book is unavailable

18.2 Design thinking

- A book should know whether it is available.
- A member should be able to issue a book.
- If the book is already issued, the program should show an exception instead of allowing wrong behavior.

18.3 Program

```
class BookNotAvailableException extends Exception {
    BookNotAvailableException(String message) {
        super(message);
    }
}

interface Issueable {
    void issueBook(Book book) throws BookNotAvailableException;
}

class Book {
    private int id;
    private String title;
    private boolean available;

    Book(int id, String title) {
        this.id = id;
        this.title = title;
        this.available = true;
```

```java
    }

    int getId() {
        return id;
    }

    String getTitle() {
        return title;
    }

    boolean isAvailable() {
        return available;
    }

    void setAvailable(boolean available) {
        this.available = available;
    }
}

class Member implements Issueable {
    private int memberId;
    private String memberName;

    Member(int memberId, String memberName) {
        this.memberId = memberId;
        this.memberName = memberName;
    }

    @Override
    public void issueBook(Book book) throws BookNotAvailableException {
        if (!book.isAvailable()) {
            throw new BookNotAvailableException("Book not available");
        }
        book.setAvailable(false);
        System.out.println(memberName + " issued " + book.getTitle());
    }

    public void issueBook(Book book, int days) throws BookNotAvailableException {
        issueBook(book);
        System.out.println("Issued for " + days + " days");
    }
}
```

18.4 What concepts are shown here?

- **Encapsulation**: private fields with controlled access
- **Constructor use**: objects initialized properly
- **Interface**: issue behavior defined by contract
- **Method overloading**: two `issueBook()` versions

- **Exception handling**: unavailable book case

19 How to Write Better Answers in the Exam

19.1 For theory questions

1. Start with one-line definition.
2. Give 2 to 4 clear points.
3. Add one small example if possible.

19.2 For difference questions

Use direct comparison points:

- meaning
- use
- limitations
- example

19.3 For code tracing or error questions

1. identify compile-time errors first
2. then identify runtime errors
3. explain why each error occurs
4. if possible, show corrected code

19.4 For program design questions

1. write class and interface names clearly
2. keep methods small and meaningful
3. make the `main()` method demonstrate all features
4. include exception handling if the question asks for it

20 Common Mistakes to Avoid

- Using `==` instead of `equals()` for string content comparison
- Treating abstract class and interface as exactly the same thing
- Forgetting that static context cannot directly access instance members
- Creating an inner class object without an outer object

- Using `throw` and `throws` interchangeably
- Writing all logic inside `main()` instead of designing proper classes
- Forgetting to override methods where required
- Ignoring exception handling in the long design questions

21 Quick Revision Notes

One-line recalls

- **Encapsulation**: keep data and methods together, protect data.
- **Abstraction**: show only essential behavior.
- **Inheritance**: child reuses parent features.
- **Polymorphism**: same interface, different behavior.
- `==`: compares references for objects.
- `equals()`: compares logical content.
- **Abstract class**: partial implementation.
- **Interface**: behavior contract.
- `this`: current object.
- `super`: immediate parent object.
- `final`: stop change / stop override / stop inheritance.
- **Static nested class**: belongs to class.
- **Inner class**: belongs to object.
- **Exception**: abnormal event during execution.
- **Custom exception**: application-specific error type.

22 Self-Check Questions

1. Why is encapsulation useful in real programs?
2. What is the exact difference between `==` and `equals()`?
3. Why does Java not support multiple inheritance of classes?
4. When should you prefer an interface over an abstract class?
5. Why does a non-static inner class need an outer object?

6. What is the difference between method overloading and overriding?
7. What is the difference between `throw` and `throws`?
8. In the bank system question, which class should override interest calculation and why?
9. In the library system question, why is exception handling important?

23 Final Advice

If you want to score well in this paper, do not memorize only definitions. Understand the relationship between concepts. For example:

- If you understand classes and objects, then encapsulation becomes natural.
- If you understand inheritance, then `super`, overriding, and abstract class questions become easier.
- If you understand static vs non-static behavior, then nested class questions become much easier.
- If you understand program flow, then exception handling becomes logical rather than confusing.

The best preparation strategy is:

1. read the concept
2. write one tiny example
3. explain the output in words
4. then practice the exam-style question